

VIBECODING MASTERCLASS 2025

Da Prompt a Produzione: Guida Pratica per l'AI Engineer

Indice

1. **Introduzione:** Il Manifesto del Vibecoding
2. **Lo Stack Tecnologico:** Gli Strumenti del Mestiere
3. **Fase 0: Strategia & Specifiche (Il Cervello)**
4. **Fase 1: Genesi Rapida con Lovable (Il Creatore)**
5. **Fase 2: Governance del Progetto (Le Regole)**
6. **Fase 3: Ingegnerizzazione via CLI (Il Braccio)**
7. **Fase 4: Quality Assurance & Security**
8. **Appendice:** Template e Prompt Pronti all'Uso

1. Introduzione: Il Manifesto del Vibecoding - L'Architettura dell'Intenzione

"Dimentica la Sintassi, Cura l'Intento"

Il Vibecoding non è semplicemente l'automazione dello *scrivere* codice tramite un'Intelligenza Artificiale. È una vera e propria **trasformazione epistemologica** nel modo in cui concepiamo e realizziamo il software. Se lo sviluppo software tradizionale è intrinsecamente paragonabile al mestiere minuzioso dello *scrivere* un libro, dove ogni parola (la sintassi) è posta manualmente in un ordine rigoroso, il Vibecoding si posiziona come l'arte di **dirigere un'opera complessa**.

In questa nuova dinamica, non sei più il manovale della tastiera, ma il **Vibe Architect**. Il tuo compito non è l'esecuzione, ma la **definizione chiara e inequivocabile dell'intento**. L'AI non è un semplice strumento, ma la tua *troupe di sviluppo* altamente competente – in grado di convertire la tua visione (il "vibe") in codice funzionale, scalabile e sicuro. Questo passaggio da esecutore a direttore massimizza il *leverage* intellettuale e sposta il valore dello sviluppatore dall'abilità di digitare velocemente alla **capacità di specificare con rigore e creatività**.

Le 3 Leggi Cardinali del Vibecoding Enterprise (La Trinità dello Sviluppo Moderno):

L'adozione del Vibecoding in un contesto enterprise non può essere casuale; richiede una disciplina ferrea, codificata nelle sue tre leggi fondamentali che garantiscono velocità, qualità e sostenibilità:

1. Prima i Documenti, poi il Codice (Requirement-Driven Design):

- **Principio:** L'unica verità operativa risiede nel documento di specifica.
- **Implementazione Pratica:** Non deve essere scritta, né accettata, una singola riga di codice da parte dell'AI o dell'umano, senza l'esistenza di un file di specifica approvato e versionato (e.g., **REQUIREMENTS.md**, **ARCHITECTURE.md**). Questo assicura che il *cosa* sia definito prima del *come*.
- **Vantaggio Enterprise:** Elimina le ambiguità, riduce il *refactoring* non necessario e garantisce che il codice generato sia un'implementazione diretta di un requisito aziendale validato, non una congettura.

2. Velocità all'inizio, Rigore alla fine (The 80/20 Rule of AI Generation):

- **Principio:** Sfrutta la potenza dell'AI per la rapida prototipazione e il *scaffolding*.
- **Implementazione Pratica:** Utilizza **strumenti visuali e interattivi** (chat, interfacce utente) per raggiungere l'80% della funzionalità in una frazione del tempo tradizionale. Tuttavia, il critico 20% – che include sicurezza, ottimizzazione delle prestazioni, e integrazione nell'infrastruttura esistente – deve essere gestito da **agenti CLI (Command Line Interface)** che operano in modo deterministico e tracciabile, con accesso a *linting*, *testing* e *security scanning* automatizzati. Questo bilanciamento garantisce che la velocità iniziale non comprometta la robustezza finale.

3. Code is Liability (Massimizzare l'Astrazione, Minimizzare il Debito):

- **Principio:** Ogni linea di codice *scritta a mano* o *generata* è un costo futuro in termini di manutenzione, debug e aggiornamento (il "debito tecnico").
- **Implementazione Pratica:** L'obiettivo primario è **ridurre al minimo** il codice che necessita di sorveglianza umana. Il paradosso del Vibecoding è che il **codice generato dall'AI deve rispettare standard di qualità intrinsecamente più alti** di quanto sia realisticamente possibile per lo sviluppatore umano. Questo include l'adesione rigorosa ai principi di ingegneria del software come **SOLID**, **DRY (Don't Repeat Yourself)** e l'immutabilità laddove appropriato. La responsabilità del Vibe Architect diventa quindi la validazione della *qualità intrinseca* del codice, non la sua scrittura.

In sintesi, il Vibecoding sposta il focus dallo sforzo manuale alla chiarezza concettuale, rendendo l'Intento la valuta più preziosa nello sviluppo software moderno.

1.

2. Lo Stack Tecnologico

Utilizzeremo una suite di strumenti selezionati per la loro sinergia.

- **Strategia:** ChatGPT o Gemini Advanced (per generare le specifiche).
- **Genesi (0 → 1):** Lovable.dev (per creare UI, Database e Backend in un click).
- **Repository:** GitHub (la fonte di verità).
- **Engineering (1 → 100):** Gemini CLI (`npm i -g @google/gemini-cli`) o OpenAI Codex CLI (agenti da terminale).

3. Fase 0: Strategia & Specifiche (Spec-Driven Development)

Il Problema: L'AI "allucina" quando le istruzioni sono vaghe.

La Soluzione: Usiamo un LLM per scrivere le specifiche tecniche prima di aprire l'editor.

Azione: Generare la "Trinità" dei Documenti

Vai su ChatGPT/Gemini e usa questo prompt per creare le fondamenta del progetto.

Prompt di Strategia:

"Agisci come un Senior Product Manager e Software Architect. Ho un'idea per un'app:.

Il tuo compito è generare 3 file Markdown distinti per guidare lo sviluppo:

1. **REQUIREMENTS.md:** Contiene le Epiche, le User Stories e i Criteri di Accettazione (sintassi Gherkin).
2. **DESIGN.md:** Definisce lo stack tecnico (React, Supabase, Tailwind), lo schema del database (tabelle, campi, relazioni) e i diagrammi di flusso testuali.
3. **TASKS.md:** Una checklist granulare di task tecnici (- [] Task 1). Ogni task deve essere abbastanza piccolo da rappresentare un singolo commit.

Infine, genera un '**God Prompt**' basato su questi documenti che io possa incollare in Lovable.dev per generare l'app."

4. Fase 1: Genesi Rapida con Lovable

Ora usiamo Lovable come un "Junior Developer velocissimo" per creare la massa critica del

progetto.

II "God Prompt" per Lovable

Incolla in Lovable il prompt generato nella fase precedente. Assicurati che includa esplicitamente:

- **Stack:** React, Tailwind, Supabase (Auth + DB).
- **Features:** Dashboard, CRUD operations, File Upload.
- **Testing:** "Genera file di test (Vitest) per la logica di business e test E2E (Playwright) per il login, anche se non puoi eseguirli."

Risultato Atteso: In 5-10 minuti avrai un'app funzionante, deployata e con il database configurato.

II Ponte (Handoff)

1. In Lovable, clicca **GitHub Sync** ed esporta il codice.
2. Clona il repo in locale:
3. Bash

```
git clone https://github.com/tuo-user/tuo-progetto.git
cd tuo-progetto
npm install
4.
```

5. Fase 2: Governance del Progetto

Prima di far lavorare l'AI sul tuo computer locale, devi darle le regole del gioco.

La Costituzione: **.cursorrules** (o **GEMINI.md**)

Crea un file nella root del progetto chiamato **.cursorrules** (se usi Cursor) o **GEMINI.md** (se usi Gemini CLI).

Copia-Incolla questo contenuto:

Project Constitution & Coding Standards

1. Core Principles (SOLID & Clean Code)

- **SRP (Single Responsibility):** Ogni file deve fare una sola cosa. Se un componente React supera le 150 righe, proponi di dividerlo.
- **DRY (Don't Repeat Yourself):** Non duplicare la logica. Estrai utility functions in `src/utils`.
- **KISS:** Preferisci la leggibilità alla complessità.

2. Tech Stack Rules

- **React:** Usa Functional Components e Hooks. Niente classi.
- **Styling:** Usa Tailwind CSS. Non creare file CSS separati se non strettamente necessario.
- **Database:** Tutte le chiamate a Supabase devono passare per `src/services/`. Mai chiamare il DB direttamente dalla UI.

3. Workflow Agentico

- **Task List:** Prima di lavorare, leggi `TASKS.md`. Esegui il primo task non spuntato.
- **TDD:** Prima di scrivere codice funzionale, verifica o crea un test che fallisce.
- **Commit:** Dopo aver completato un task, aggiorna `TASKS.md` (- [x]) ed esegui il commit.

6. Fase 3: Ingegnerizzazione via CLI

Qui trasformiamo il prototipo in prodotto professionale usando l'AI da riga di comando.

Installazione Strumenti

- **Gemini CLI:** `npm install -g @google/gemini-cli` (Gratis, contesto enorme).
- **Codex CLI:** `npm install -g @openai/codex` (Logica precisa, richiede sub).

Missione 1: Il Loop "Task-Execute-Commit"

Non dire all'AI cosa fare ogni volta. Dille di seguire il piano.

Comando (Terminale):

Bash

`gemini -p "Agisci come un Senior Developer.`

`1. Leggi 'TASKS.md'.`

`2. Identifica il primo task non completato.`

3. Leggi 'DESIGN.md' per il contesto.
 4. Implementa il task modificando i file necessari.
 5. Esegui i test ('npm test') per verificare.
 6. Se i test passano, aggiorna 'TASKS.md' segnando il task come fatto [x].
 7. Esegui 'git commit -am \"Feat: completed task X\"'.
- Procedi in autonomia." --yolo

(Nota: *--yolo* permette all'agente di eseguire comandi senza chiedere conferma costante).

Missione 2: Refactoring SOLID

Lovable tende a creare componenti enormi ("God Objects"). Usiamo la CLI per pulirli.

Comando:

Bash

gemini "Analizza 'src/pages/Dashboard.tsx'.

Viola il principio SRP (Single Responsibility).

1. Estrai la logica di fetch dati in un hook 'hooks/useDashboardData.ts'.
2. Estrai i grafici in componenti isolati in 'components/charts/'.
3. Assicurati che l'app compili ancora.
4. Committa con 'Refactor: Applied SOLID to Dashboard'."

7. Fase 4: Quality Assurance & Security

Riparazione Automatica dei Test

I test generati da Lovable spesso falliscono appena scaricati.

Comando:

Bash

gemini -p "Esegui 'npm test'. Leggi gli errori nel terminale.

Analizza i file di test e il codice sorgente.

Correggi il codice o i test finché non passano tutti (GREEN).

Non fermarti finché la suite non è verde." --yolo

Security Audit

Comando:

Bash

gemini "Scansiona la cartella 'supabase/migrations'.

Verifica che ogni tabella abbia la Row Level Security (RLS) abilitata.

Se trovi una policy che permette l'accesso pubblico indiscriminato, segnalala in un file

'SECURITY_WARNINGS.md'."

8. Appendice: Cheat Sheet Comandi

Obiettivo	Comando/Prompt
Generare Spec	(Su ChatGPT) "Genera REQUIREMENTS.md, DESIGN.md, TASKS.md per..."
Pulire Codice	"Rifattorizza questo file applicando DRY e SRP."
Fix Test	"Esegui i test, leggi gli errori e correggi il codice."
Aggiornare Task	"Leggi TASKS.md, completa il prossimo item e spuntalo."